

OPPU and SPRInG: The Two Closest Per-User Training Methods

Technical Deep Dive

August 25, 2026

Abstract

We seek one policy per user, trained first from that user’s thoughts, conversations, and task history, then updated from later interactions. OPPU is the closest offline method: it fits one private adapter from one user’s history. SPRInG is the closest continual method: it updates that adapter from chronological interaction batches and retains a bounded personal memory. This note specifies their workflows, objectives, data models, inputs, outputs, and the exact point where our agent-learning problem begins.

1 The Two Closest Per-User Training Methods

1.1 What counts as a close match

Our unit of personalization is one user. A deployment run owns that user’s adapter, optimizer state, memory, and checkpoint history. The initial policy is trained from the user’s thoughts, conversations, corrections, and task records. Later interactions update the same policy. A shared backbone may be trained separately, but the target user’s raw history does not train another user’s deployed branch.

Under this definition, the two closest papers are OPPU [Tan et al., 2024] and SPRInG [Kim and Kim, 2026]. OPPU is the closest offline initializer: it trains one adapter from one user’s history. SPRInG is the closest continual method: it carries that user’s adapter forward and updates it from chronological interaction batches. Together they bracket our offline-to-online workflow.

We exclude methods that train one policy across many users and personalize only through a profile or prompt. They do not produce the user-owned policy studied here.

1.2 OPPU: one history, one adapter

Model and data. OPPU freezes a task-adapted base model with parameters θ and assigns user u a private PEFT module $\Delta\theta_u$. The deployed model is

$$M_u = M_{\theta \oplus \Delta\theta_u}. \quad (1)$$

The user’s history $H_u^{<t}$ contains only records observed before the current query. A record is either a supervised pair (x_i, y_i) or a plain text sequence x_i . The first form covers prior requests and accepted responses. The second can hold notes, conversation text, or other user-authored material.

Workflow.

1. Adapt a shared base model to the task using users outside the target cohort, then merge the task LoRA into the base.
2. Freeze the shared base.

Table 1: Why OPPU and SPRInG are the closest matches.

Property	OPPU	SPRInG	Our setting
Policy owner	One PEFT module per user	One evolving LoRA adapter per user	One user-owned policy branch
Historical input	Paired behaviors or plain text	Ordered query–response records	Thoughts, conversations, task trajectories, and outcomes
Offline update	Cross-entropy or next-token training	Pretrained period-0 adapter	User-history initialization
Later update	None	Selective SFT on each new period	Preference or outcome-driven online update
Persistent output	Personal adapter	Adapter lineage and bounded memory	Adapter, optimizer state, memory, and audit receipt

Algorithm 1: OPPU for one user

Input: Frozen task model M_θ ; chronological user history H_u ; optional retriever R
Output: Personal adapter $\Delta\theta_u$

- 1 initialize $\Delta\theta_u$
- 2 **foreach** record $h_i \in H_u$ in chronological order **do**
- 3 construct its target from (x_i, y_i) or right-shifted text x_i
- 4 if retrieval is enabled, retrieve only from $H_u^{<i}$
- 5 accumulate the loss in [equation \(2\)](#) or [equation \(3\)](#)
- 6 update only $\Delta\theta_u$ for a fixed number of epochs
- 7 save $\Delta\theta_u$ with its user and history-version identifiers

3. Initialize one LoRA module $\Delta\theta_u$ for the target user.
4. Train it only on H_u . When retrieval is enabled, each example may retrieve only records that preceded that example.
5. Save $\Delta\theta_u$. At inference, load the frozen base and this user’s adapter, then answer the new query.

Optimization. For paired history, OPPU minimizes conditional cross-entropy:

$$\mathcal{L}_{\text{pair}}(\Delta\theta_u) = - \sum_{(x_i, y_i) \in H_u} \sum_{j=1}^{|y_i|} \log p_{\theta, \Delta\theta_u}(y_{i,j} \mid \varphi(x_i), y_{i,<j}). \quad (2)$$

When history contains plain text only, the target is the next token:

$$\mathcal{L}_{\text{text}}(\Delta\theta_u) = - \sum_{x_i \in H_u} \sum_{j=1}^{|x_i|} \log p_{\theta, \Delta\theta_u}(x_{i,j} \mid x_{i,<j}). \quad (3)$$

The paper uses rank-8 LoRA on the query and value projections, learning rate 10^{-5} , and two or three fixed epochs. It does not report an online update, replay rule, rollback rule, or optimizer-state lifecycle.

Inputs and outputs. Training consumes a frozen base model, one user identifier, and that user’s historical records. It outputs one loadable adapter. Inference consumes the adapter and a new query; optional retrieval or a generated profile may also enter the prompt. The paper evaluates Llama-2-7B on seven LaMP tasks using the 100 users with the longest histories.

What transfers. OPPU gives us the correct ownership boundary and a practical offline initializer. It does not learn from task outcomes, pairwise preferences, or live feedback. It also treats each record as text rather than as a replayable agent trajectory. Our offline stage can use its per-user adapter design, but our record schema must retain actions, tools, outcomes, timestamps, and the generating checkpoint.

1.3 SPRInG: one adapter through time

Model and data. SPRInG starts from the same ownership model. User u has a frozen backbone M_θ , an adapter $\phi_u^{(t)}$, and a bounded memory $B_u^{(t)}$. New interactions arrive in periods:

$$D_u^{(t)} = \{(q_i, a_i)\}_{i=1}^{n_t}. \quad (4)$$

Period 0 supplies the initial adapter. Each later period updates that same adapter. The paper uses text-generation records, not tool trajectories or explicit rewards.

Workflow.

1. Score every new (q, a) under the frozen base and the current user-adapted model.
2. Keep the top 30% by drift score.
3. Copy $\phi_u^{(t-1)}$ and fine-tune the copy on the selected records.
4. Merge selected records with the old memory, score them again under the updated adapter, and retain the hardest 50.
5. For a new query, retrieve two memory records. Use them only when their similarity passes a fixed gate.
6. Decode by interpolating adapter-only and retrieval-conditioned token probabilities.

Selection and update. Let

$$p_{\text{base}}(q, a) = P(a \mid q; M_\theta), \quad (5)$$

$$p_{\text{adap}}^{(t-1)}(q, a) = P(a \mid q; M_{\theta, \phi_u^{(t-1)}}). \quad (6)$$

SPRInG scores a record by

$$S_t(q, a) = -\log \frac{p_{\text{adap}}^{(t-1)}(q, a)}{p_{\text{base}}(q, a)} + \alpha \log p_{\text{base}}(q, a). \quad (7)$$

The likelihood ratio favors behavior not yet captured by the adapter. The base likelihood suppresses malformed or low-quality text. If τ_p is the period’s percentile threshold,

$$D_{\text{drift}}^{(t)} = \{(q, a) \in D_u^{(t)} : S_t(q, a) \geq \tau_p\}. \quad (8)$$

The adapter then takes supervised next-token updates on $D_{\text{drift}}^{(t)}$ only:

$$\phi_u^{(t)} = \text{SFT} \left(\phi_u^{(t-1)}, D_{\text{drift}}^{(t)} \right). \quad (9)$$

Algorithm 2: SPRInG for one user

Input: Frozen M_θ ; initial $\phi_u^{(0)}$; stream $\{D_u^{(t)}\}_{t \geq 1}$; percentile p ; budget $N_{\max}$
Output: Adapter lineage $\{\phi_u^{(t)}\}$ and memories $\{B_u^{(t)}\}$

```
1  $B_u^{(0)} \leftarrow \emptyset$ 
2 for  $t = 1, 2, \dots$  do
3   score  $D_u^{(t)}$  with equation \(7\)
4   retain  $D_{\text{drift}}^{(t)}$  with equation \(8\)
5    $\phi_u^{(t)} \leftarrow \text{SFT}(\phi_u^{(t-1)}, D_{\text{drift}}^{(t)})$ 
6    $C_u^{(t)} \leftarrow D_{\text{drift}}^{(t)} \cup B_u^{(t-1)}$ 
7   rescore  $C_u^{(t)}$  under  $\phi_u^{(t)}$ 
8   retain  $B_u^{(t)}$  with equation \(11\)
```

Residual memory and inference. After the update, SPRInG rescans

$$C_u^{(t)} = D_{\text{drift}}^{(t)} \cup B_u^{(t-1)} \quad (10)$$

and recomputes [equation \(7\)](#) with $\phi_u^{(t)}$. It keeps the $N_{\max}$ highest-scoring records:

$$B_u^{(t)} = \text{Top}_{N_{\max}} \{(q, a) \in C_u^{(t)}; S'_t(q, a)\}. \quad (11)$$

For a test query, the same adapted model produces an adapter-only distribution p_{par} and, when memory passes the relevance gate, a retrieval-conditioned distribution p_{ret} . The final distribution is

$$p_{\text{fin}} = \lambda p_{\text{ret}} + (1 - \lambda) p_{\text{par}}. \quad (12)$$

Memory affects inference only. It is not replayed into later optimizer steps.

Inputs and outputs. Training consumes one user’s initial adapter and a chronological stream of query–response batches. Each period outputs a new adapter and memory snapshot. Inference consumes the current adapter, current memory, and a new query, and outputs a response. The main experiment uses 100 LongLaMP users, five chronological periods, Gemma-3-4B-IT, rank-8 LoRA, AdamW, two epochs, and a 90/10 update/evaluation split within each period.

What transfers. SPRInG is the closest end-to-end prior because it preserves user ownership through time. Its main gap is the learning signal: difficult-to-predict text is treated as preference drift, and updates still maximize likelihood. Our agent receives richer evidence. A completed task can include tool calls, environment outcomes, user corrections, and explicit preferences. Those signals should determine eligibility and reward; likelihood novelty alone cannot tell whether an action helped the user.

1.4 Concrete design for our policy

The two papers suggest a clean baseline for our candidate online updates:

1. Serialize user u ’s thoughts, conversations, and successful task records into a chronological offline history.
2. Train a user-owned adapter with the OPPU objective. Save the adapter, optimizer state, data boundary, and base-model hash as checkpoint $c_u^{(0)}$.

3. Collect later tasks only with the active user checkpoint. Preserve states, actions, tools, outcomes, corrections, and behavior log-probabilities.
4. Use a SPRInG-style selection stage, but score evidence by task validity, preference information, and provenance rather than likelihood novelty alone.
5. Apply the chosen update from this note. Activate it only after the shared validation and rollback gate.

This baseline separates inheritance from novelty. OPPU supplies the offline per-user policy. SPRInG supplies the chronological adapter lineage. Our contribution begins where supervised text prediction ends: agent evidence, preference- or outcome-driven updates, auditable optimizer state, and safe activation.

References

- Seoyeon Kim and Jaehyung Kim. SPRInG: Continual LLM personalization via selective parametric adaptation and retrieval-interpolated generation. *arXiv preprint arXiv:2601.09974*, 2026. URL <https://arxiv.org/abs/2601.09974>.
- Zhaoxuan Tan, Qingkai Zeng, Yijun Tian, Zheyuan Liu, Bing Yin, and Meng Jiang. Democratizing large language models via personalized parameter-efficient fine-tuning. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024. doi: 10.18653/v1/2024.emnlp-main.372. URL <https://arxiv.org/abs/2402.04401>.